



Week 01 — Getting Started with R

From wide to long, grouping and summarizing, measures of central tendency and variability, and a first pass at ggplot2

AUTHORS

Johann-Friedrich Salzmänn  

Joshua Elias Schmidt Rodrigues  

DATE

2026-09-09

Topics:

Package management	2
Where we left off	2
Warm-up: from wide to long	2
Warm-up: grouping and summarizing	3
Warm-up: plotting the result	5
Measures of central tendency: the mean	6
Central tendency and variability, the rest of the toolbox	8
The mode: three definitions	8
Plotting data, revisited in ggplot2	11
Try it: AfD's historic high and low	14
Try it: how much did the polls agree right before the 2025 election?	16
Try it: BSW's rise above — and fall back below — the 5% threshold	18

Package management

Same as every session: bootstrap pacman, then load the tidyverse.

```
if (!require(pacman)) install.packages("pacman")
```

MESSAGE

Loading required package: pacman

```
p_load(tidyverse)
```

Where we left off

R/RStudio/Git, objects, data structures (vectors, matrices, data frames, arrays, lists), indexing/subsetting, and loading data are all Intro R course material — see the [Intro R course repository](#) (Day 1 for objects/data structures/indexing, Day 3 for loading data formats). We pick up straight from real, already-familiar data.

Warm-up: from wide to long

The same Bundestag polling data from the Intro R course, still wide (one column per party) — originally sourced from [ZEIT ONLINE's polling tracker](#).

```
polls <- read_csv("data/polls.csv")
```

MESSAGE

Rows: 2077 Columns: 12

— Column specification —

Delimiter: ","

chr (3): Poll_ID, Pollster, Period

dbl (8): n, CDU/CSU, SPD, Gruene, FDP, Linke, AfD, BSW

date (1): Date

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
polls |> head(100)
```

> OUTPUT

```
# A tibble: 100 × 12
  Poll_ID Pollster Date n Period CDU CSU SPD Gruene FDP Linke
  <chr> <chr> <date> <dbl> <chr> <dbl> <dbl> <dbl> <dbl> <dbl>
1 BT021701040 Forsa 2017-01-04 2001 27.12... 37 21 10 6 9
2 BT01170105P Infrates... 2017-01-05 1505 02.01... 37 20 9 5 9
3 BT03170107L Verian 2017-01-07 834 03.01... 38 22 10 5 9
4 BT02170111P Forsa 2017-01-11 2501 02.01... 37 20 10 6 9
5 BT03170114M Verian 2017-01-14 1878 06.01... 37 21 10 6 10
6 BT02170118Q Forsa 2017-01-18 2503 09.01... 38 21 9 6 9
7 BT03170121N Verian 2017-01-21 2805 12.01... 36 21 9 6 11
8 BT02170125R Forsa 2017-01-25 2504 16.01... 37 21 10 6 9
9 BT01170127Q Infrates... 2017-01-27 1029 24.01... 35 23 9 6 8
10 BT03170128O Verian 2017-01-28 1996 19.01... 37 23 10 6 10
# i 90 more rows
# i 2 more variables: AfD <dbl>, BSW <dbl>
```

```
polls_long <- polls |>
  pivot_longer(CDU CSU:BSW, names_to = "party", values_to = "share")
polls_long
```

> OUTPUT

```
# A tibble: 14,539 × 7
  Poll_ID Pollster Date n Period party share
  <chr> <chr> <date> <dbl> <chr> <chr> <dbl>
1 BT021701040 Forsa 2017-01-04 2001 27.12.-30.12. CDU CSU 37
2 BT021701040 Forsa 2017-01-04 2001 27.12.-30.12. SPD 21
3 BT021701040 Forsa 2017-01-04 2001 27.12.-30.12. Gruene 10
4 BT021701040 Forsa 2017-01-04 2001 27.12.-30.12. FDP 6
5 BT021701040 Forsa 2017-01-04 2001 27.12.-30.12. Linke 9
6 BT021701040 Forsa 2017-01-04 2001 27.12.-30.12. AfD 12
7 BT021701040 Forsa 2017-01-04 2001 27.12.-30.12. BSW NA
8 BT01170105P Infratest dimap 2017-01-05 1505 02.01.-04.01. CDU CSU 37
9 BT01170105P Infratest dimap 2017-01-05 1505 02.01.-04.01. SPD 20
10 BT01170105P Infratest dimap 2017-01-05 1505 02.01.-04.01. Gruene 9
# i 14,529 more rows
```

Warm-up: grouping and summarizing

Grouping by two variables at once, same idea as the toy `cat1/cat2` tibble from Intro R Day 3 — one row per party x year here.

```
polls_long |>
  mutate(year = year(Date)) |>
  group_by(party, year) |>
  summarize(avg_share = mean(share, na.rm = TRUE)) |>
  ungroup() |>
  arrange(party, year)
```

MESSAGE

`summarise()` has regrouped the output.
 i Summaries were computed grouped by party and year.
 i Output is grouped by party.
 i Use `summarise(.groups = "drop\_last")` to silence this message.
 i Use `summarise(.by = c(party, year))` for per-operation grouping
 (`?dplyr::dplyr\_by`) instead.

> OUTPUT

```
# A tibble: 70 × 3
  party   year avg_share
  <chr> <dbl>   <dbl>
1 AfD    2017     9.90
2 AfD    2018    14.2
3 AfD    2019    13.2
4 AfD    2020    10.4
5 AfD    2021    10.4
6 AfD    2022    11.7
7 AfD    2023    18.4
8 AfD    2024    18.2
9 AfD    2025    23.8
10 AfD    2026    26.4
# i 60 more rows
```

Collapsing to one row per party only.

```
party_avg <- polls_long |>
  group_by(party) |>
  summarize(avg_share = mean(share, na.rm = TRUE)) |>
  arrange(desc(avg_share))
party_avg
```

> OUTPUT

```
# A tibble: 7 × 2
  party   avg_share
  <chr>   <dbl>
1 CDU/CSU 29.0
2 SPD     17.7
3 AfD     15.7
4 Grüne   15.5
5 Linke   7.04
6 FDP     6.93
7 BSW     5.09
```

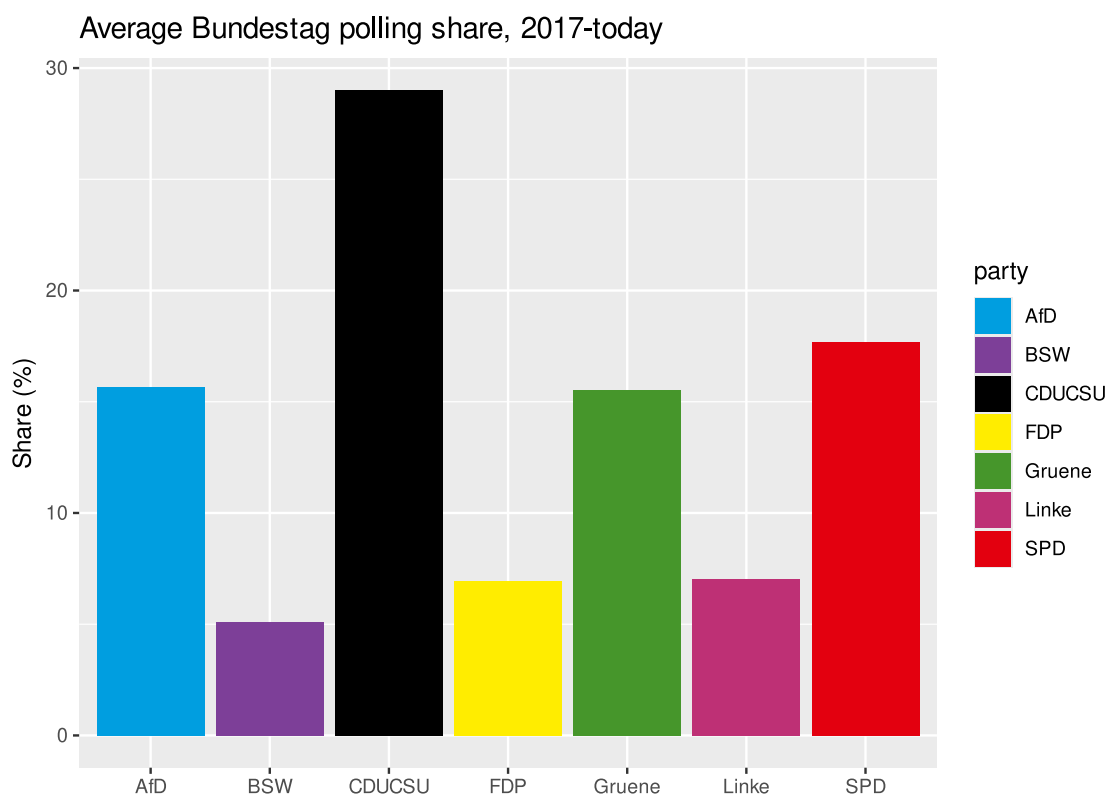
Warm-up: plotting the result

Real party colors, same vector as Intro R Day 2/3.

```
party_colors <- c(  
  CDU = "#000000",  
  SPD  = "#E3000F",  
  Gruene = "#46962b",  
  FDP  = "#ffed00",  
  Linke = "#be3075",  
  AfD  = "#009ee0",  
  BSW  = "#7d3f98"  
)
```

A bar chart of the `summarize()` result.

```
party_avg |>  
  ggplot(aes(x = party, y = avg_share, fill = party)) +  
  geom_col() +  
  scale_fill_manual(values = party_colors) +  
  labs(title = "Average Bundestag polling share, 2017-today", x = NULL, y = "Share (%)")
```



And the full trend line — exactly Intro R Day 2/3's central plot.

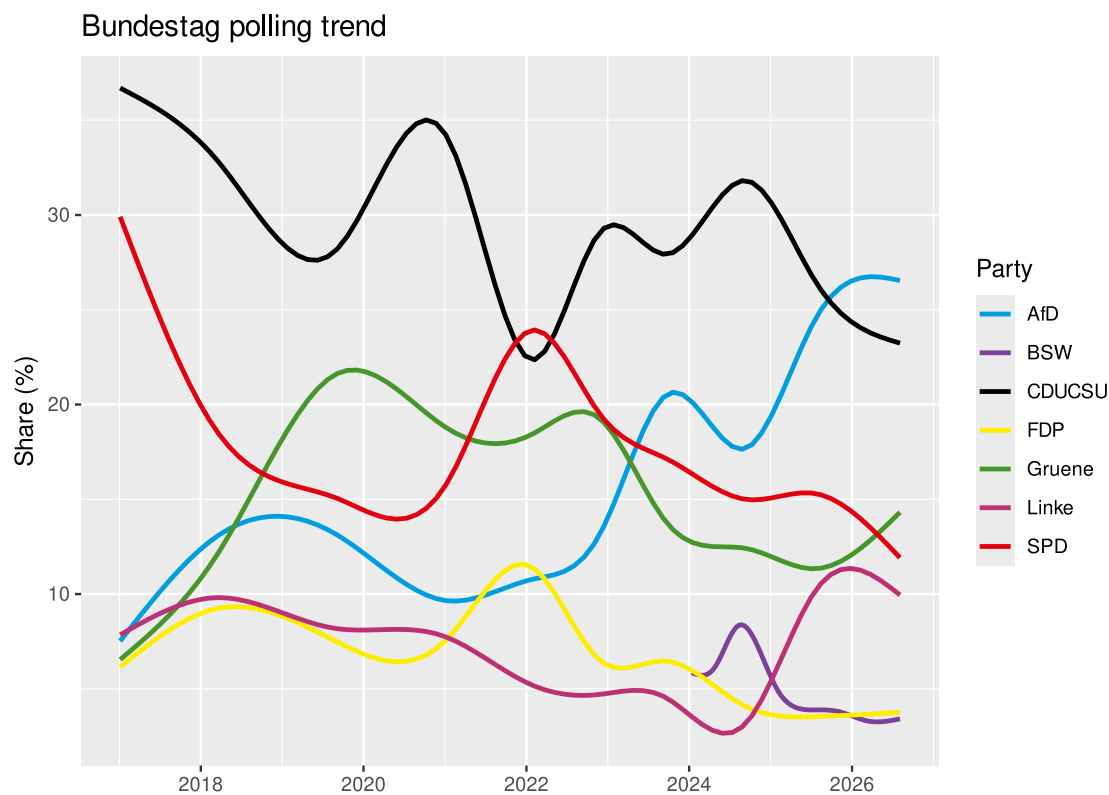
```
polls_long |>
  ggplot(aes(x = Date, y = share, color = party)) +
  geom_smooth(se = FALSE) +
  scale_color_manual(values = party_colors) +
  labs(title = "Bundestag polling trend", x = NULL, y = "Share (%)", color = "Party")
```

MESSAGE

`geom\_smooth()` using method = 'gam' and formula = 'y ~ s(x, bs = "cs")'

WARNING

Warning: Removed 1548 rows containing non-finite outside the scale range (`stat\_smooth()`).



Measures of central tendency: the mean

```
x <- 1:5
x
```

OUTPUT

```
[1] 1 2 3 4 5
```

```
mean(x)
```

>_ OUTPUT

```
[1] 3
```

```
sum(x) / length(x)
```

>_ OUTPUT

```
[1] 3
```

This only makes sense for numeric vectors.

```
x_char <- c("a", "b", "c", "d", "e")
mean(x_char)
```

⚠ WARNING

Warning in mean.default(x_char): argument is not numeric or logical: returning NA

>_ OUTPUT

```
[1] NA
```

But it does work for logicals — **TRUE** counts as 1, **FALSE** as 0.

```
x_logical <- c(TRUE, FALSE, TRUE, TRUE, FALSE)
mean(x_logical)
```

>_ OUTPUT

```
[1] 0.6
```

```
sum(x_logical)
```

>_ OUTPUT

```
[1] 3
```

The same mean again, now via a tibble and `summarize()` — from here on we keep working with tibbles for every aggregation.

```
x_tbl <- tibble(xcol = x)
x_tbl |> summarize(mean_x = mean(xcol))
```

➤ OUTPUT

```
# A tibble: 1 × 1
  mean_x
  <dbl>
1      3
```

Central tendency and variability, the rest of the toolbox

Mean: `mean()`. Median: `median()`. Variance: `var()`. Standard deviation: `sd()`. Range: `range()`. Interquartile range: `IQR()`. Pooling every party's share into one number is illustrative only — a real analysis would do this per party, as in the Try it below.

```
polls_long |>
  summarize(
    mean_share = mean(share, na.rm = TRUE),
    median_share = median(share, na.rm = TRUE),
    var_share = var(share, na.rm = TRUE),
    sd_share = sd(share, na.rm = TRUE),
    min_share = min(share, na.rm = TRUE),
    max_share = max(share, na.rm = TRUE),
    iqr_share = IQR(share, na.rm = TRUE)
  )
```

➤ OUTPUT

```
# A tibble: 1 × 7
  mean_share median_share var_share sd_share min_share max_share iqr_share
  <dbl>         <dbl>      <dbl>   <dbl>   <dbl>     <dbl>   <dbl>
1    14.9         13      73.2    8.56     2        40    12.5
```

The mode: three definitions

R has no dedicated `mode()` function. One small tibble to work with — `shares` (numeric): 3 and 7 tied at 3 occurrences each, everyone else a singleton. `parties` (character), same length: SPD is the single clear winner, no tie.


```
mode_data <- tibble(
  shares = c(3, 7, 3, 9, 7, 2, 3, 7, 5, 1),
  parties = c("SPD", "CDUCSU", "SPD", "Gruene", "SPD", "AfD", "SPD", "Linke", "FDP",
"CDUCSU")
)
mode_data
```

> OUTPUT

```
# A tibble: 10 × 2
  shares parties
  <dbl> <chr>
1     3 SPD
2     7 CDUCSU
3     3 SPD
4     9 Gruene
5     7 SPD
6     2 AfD
7     3 SPD
8     7 Linke
9     5 FDP
10    1 CDUCSU
```

`select()` the column we need, then `group_by()` + `summarize(n())` — the way we already know.

```
shares_counts <- mode_data |>
  select(shares) |>
  group_by(shares) |>
  summarize(n = n()) |>
  ungroup()
shares_counts
```

> OUTPUT

```
# A tibble: 6 × 2
  shares      n
  <dbl> <int>
1     1     1
2     2     1
3     3     3
4     5     1
5     7     3
6     9     1
```

`count()` is a shortcut for that same `group_by()+summarize(n())+ungroup()` combination, in one step — used here for `parties`.

```
parties_counts <- mode_data |>
  select(parties) |>
  count(parties)
parties_counts
```

>_ OUTPUT

```
# A tibble: 6 × 2
  parties      n
  <chr>    <int>
1 AfD          1
2 CDU/CSU      2
3 FDP           1
4 Gruene        1
5 Linke         1
6 SPD           4
```

Definition 1: every value tied for the highest count (multimodal).

```
shares_counts |>
  filter(n == max(n))
```

>_ OUTPUT

```
# A tibble: 2 × 2
  shares      n
  <dbl> <int>
1      3      3
2      7      3
```

Definition 2: a single “the” mode. `filter()`-ing for the max count alone isn’t safe here — we can’t know in advance whether there’s a tie, and if there were, every tied row would come back. `arrange()` + `slice(1)` always returns exactly one row, tie or not.

```
parties_counts |>
  arrange(desc(n)) |>
  slice(1)
```

>_ OUTPUT

```
# A tibble: 1 × 2
  parties      n
  <chr>    <int>
1 SPD           4
```

Definition 3: for numeric data, average the tied values into one number.

```
shares_counts |>
  filter(n == max(n)) |>
  summarize(mean_mode = mean(shares))
```

> OUTPUT

```
# A tibble: 1 × 1
  mean_mode
  <dbl>
1         5
```

Plotting data, revisited in ggplot2

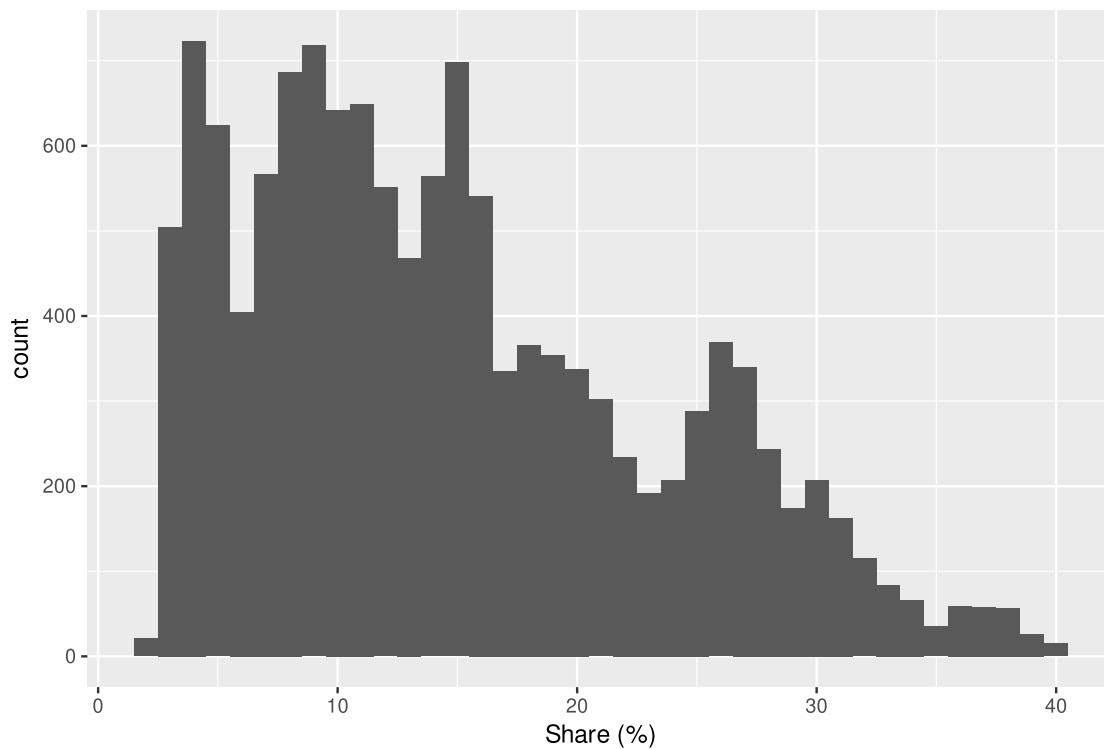
Same ideas as Intro R Day 2, quickly, plus one new geom — the bivariate scatter is skipped here, since the warm-up trend line above already showed share over time.

```
polls_long |>
  ggplot(aes(x = share)) +
  geom_histogram(binwidth = 1) +
  labs(title = "Distribution of polled party shares", x = "Share (%)")
```

⚠ WARNING

Warning: Removed 1548 rows containing non-finite outside the scale range (`stat\_bin()`).

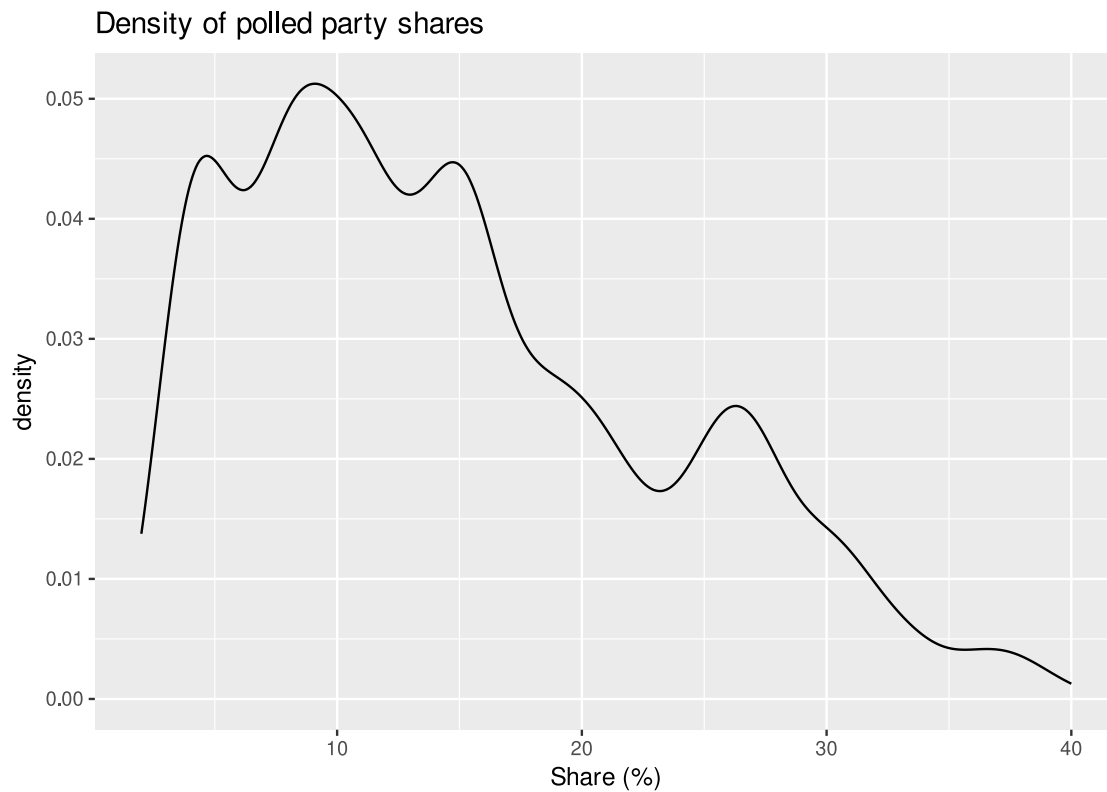
Distribution of polled party shares



```
polls_long |>
  ggplot(aes(x = share)) +
  geom_density() +
  labs(title = "Density of polled party shares", x = "Share (%)")
```

⚠ WARNING

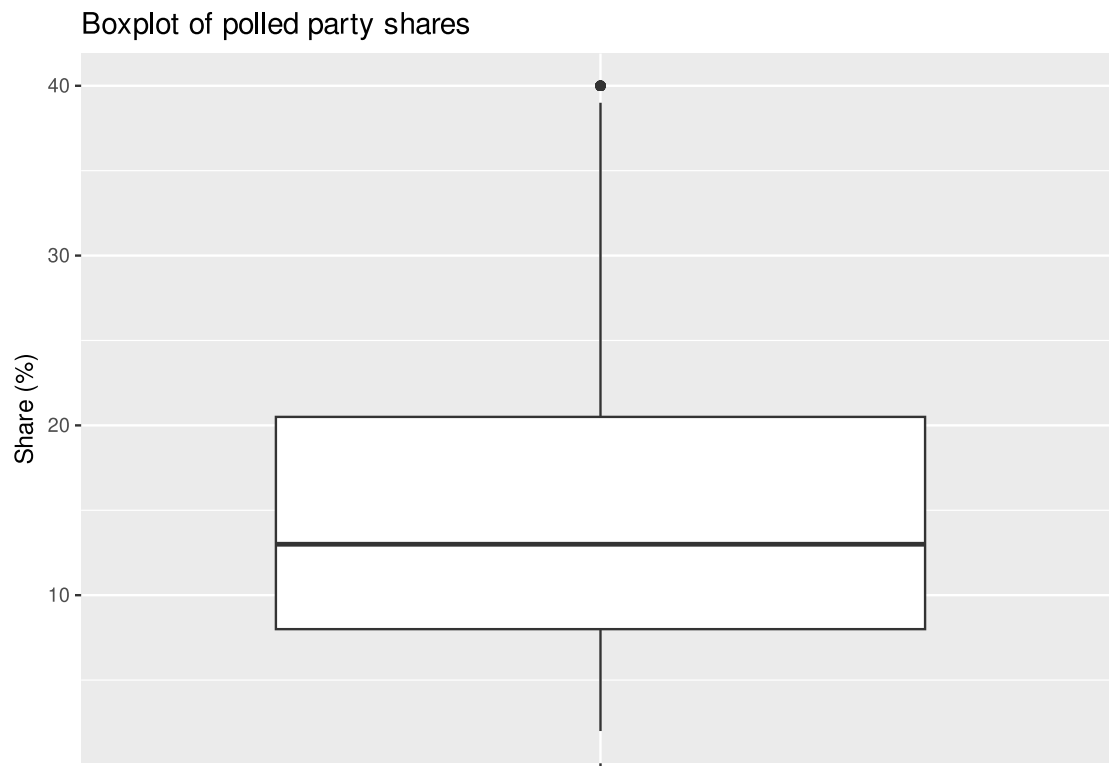
Warning: Removed 1548 rows containing non-finite outside the scale range (`stat\_density()`).



```
polls_long |>
  ggplot(aes(x = "", y = share)) +
  geom_boxplot() +
  labs(title = "Boxplot of polled party shares", x = NULL, y = "Share (%)")
```

⚠ WARNING

Warning: Removed 1548 rows containing non-finite outside the scale range (`stat\_boxplot()`).

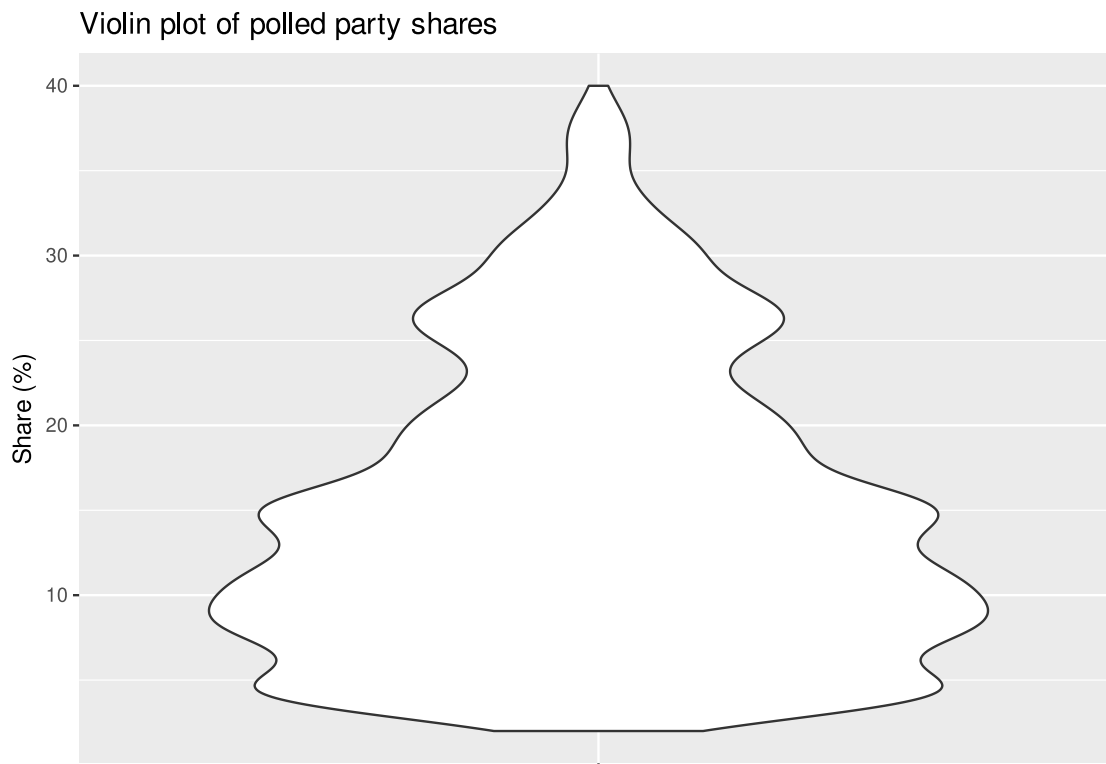


A violin plot shows the full shape of the distribution, not just its quartiles.

```
polls_long |>
  ggplot(aes(x = "", y = share)) +
  geom_violin() +
  labs(title = "Violin plot of polled party shares", x = NULL, y = "Share (%)")
```

⚠ WARNING

Warning: Removed 1548 rows containing non-finite outside the scale range (`stat\_ydensity()`).



Try it: AfD's historic high and low

Your task: populist parties' support can swing dramatically over time — looking at AfD's entire historic polling record (not just recent numbers), what's the highest and lowest share it has ever been polled at?

```
afd <- polls_long |>
  filter(party == "AfD") |>
  arrange(desc(share))
afd |> slice(1)      # highest
```

➤ OUTPUT

```
# A tibble: 1 × 7
  Poll_ID Pollster Date      n Period      party share
  <chr>    <chr>   <date>  <dbl> <chr>      <chr> <dbl>
1 BT05260516M INSA    2026-05-16 1203 11.05.–15.05. AfD      29
```

```
afd |> slice_tail(n = 1) # lowest
```

➤ OUTPUT

```
# A tibble: 1 × 7
  Poll_ID Pollster Date      n Period      party share
  <chr>    <chr>   <date>  <dbl> <chr>      <chr> <dbl>
1 BT04220531B GMS     2022-05-31 1005 25.05.–30.05. AfD      7
```

```
# filter() with the extremes works just as well
afd |> filter(share == max(share))
```

> OUTPUT

```
# A tibble: 19 × 7
  Poll_ID Pollster Date      n Period      party share
  <chr>    <chr>   <date>   <dbl> <chr>    <chr> <dbl>
1 BT05260516M INSA    2026-05-16 1203 11.05.-15.05. AfD      29
2 BT05260519N INSA    2026-05-19 2005 15.05.-18.05. AfD      29
3 BT05260523O INSA    2026-05-23 1203 18.05.-22.05. AfD      29
4 BT05260526P INSA    2026-05-26 2010 22.05.-26.05. AfD      29
5 BT05260530Q INSA    2026-05-30 1205 26.05.-29.05. AfD      29
6 BT05260602R INSA    2026-06-02 2006 29.05.-01.06. AfD      29
7 BT05260606S INSA    2026-06-06 1206 01.06.-05.06. AfD      29
8 BT05260609T INSA    2026-06-09 2009 05.06.-08.06. AfD      29
9 BT05260613U INSA    2026-06-13 1203 08.06.-12.06. AfD      29
10 BT05260616V INSA    2026-06-16 2003 12.06.-15.06. AfD      29
11 BT13260616X YouGov  2026-06-16 1793 12.06.-15.06. AfD      29
12 BT05260620W INSA    2026-06-20 1204 15.06.-19.06. AfD      29
13 BT05260623X INSA    2026-06-23 2008 19.06.-22.06. AfD      29
14 BT05260627Y INSA    2026-06-27 1204 22.06.-26.06. AfD      29
15 BT05260630A INSA    2026-06-30 2006 26.06.-29.06. AfD      29
16 BT05260704B INSA    2026-07-04 1205 29.06.-03.07. AfD      29
17 BT05260707C INSA    2026-07-07 2010 03.07.-06.07. AfD      29
18 BT05260718F INSA    2026-07-18 1201 13.07.-17.07. AfD      29
19 BT05260721G INSA    2026-07-21 2005 17.07.-20.07. AfD      29
```

```
afd |> filter(share == min(share))
```

> OUTPUT

```
# A tibble: 21 × 7
  Poll_ID Pollster Date      n Period      party share
  <chr>    <chr>   <date>   <dbl> <chr>    <chr> <dbl>
1 BT02170329B Forsa    2017-03-29 2501 20.03.-24.03. AfD       7
2 BT02170510H Forsa    2017-05-10 2004 02.05.-05.05. AfD       7
3 BT02170517I Forsa    2017-05-17 2506 08.05.-12.05. AfD       7
4 BT02170523J Forsa    2017-05-23 2502 15.05.-19.05. AfD       7
5 BT02170531K Forsa    2017-05-31 2001 22.05.-26.05. AfD       7
6 BT02170607L Forsa    2017-06-07 2501 29.05.-02.06. AfD       7
7 BT02170613M Forsa    2017-06-13 2004 06.06.-09.06. AfD       7
8 BT02170621N Forsa    2017-06-21 2505 12.06.-16.06. AfD       7
9 BT13170623H YouGov  2017-06-23 1523 19.06.-22.06. AfD       7
10 BT02170628O Forsa    2017-06-28 2502 19.06.-23.06. AfD       7
# i 11 more rows
```

Your task: different pollsters can paint quite different pictures — repeat this, but separately for each pollster: find AfD’s highest and lowest recorded share (and the date) per pollster.

```
# slice() is group-aware once grouped – same "verbs act per group" idea
# as Day 3, just applied to slice() this time. Shares are whole numbers,
# so ties are common: a trailing slice(1) always narrows a tie down to
# one representative row per pollster
```

```
afd |> # highest per pollster
  group_by(Pollster) |>
  arrange(desc(share)) |>
  slice(1)
```

> OUTPUT

```
# A tibble: 8 × 7
# Groups:   Pollster [8]
  Poll_ID Pollster Date n Period party share
<chr> <chr> <date> <dbl> <chr> <chr> <dbl>
1 BT09260624B Allensbach 2026-06-24 1047 05.06.–17.06. AfD 28
2 BT02260519F Forsa 2026-05-19 2000 12.05.–18.05. AfD 28
3 BT06260619I Forschungsgruppe Wahlen 2026-06-19 1190 16.06.–18.06. AfD 28
4 BT04260720J GMS 2026-07-20 1005 15.07.–20.07. AfD 28
5 BT05260516M INSA 2026-05-16 1203 11.05.–15.05. AfD 29
6 BT01260507I Infratest dimap 2026-05-07 1303 04.05.–06.05. AfD 27
7 BT03260603U Verian 2026-06-03 1454 27.05.–02.06. AfD 27
8 BT13260616X YouGov 2026-06-16 1793 12.06.–15.06. AfD 29
```

```
afd |> # lowest per pollster
  group_by(Pollster) |>
  filter(share == min(share)) |>
  slice(1)
```

> OUTPUT

```
# A tibble: 8 × 7
# Groups:   Pollster [8]
  Poll_ID Pollster Date n Period party share
<chr> <chr> <date> <dbl> <chr> <chr> <dbl>
1 BT09200424D Allensbach 2020-04-24 1152 01.04.–15.04. AfD 9
2 BT02170329B Forsa 2017-03-29 2501 20.03.–24.03. AfD 7
3 BT06200424G Forschungsgruppe Wahlen 2020-04-24 1323 20.04.–23.04. AfD 9
4 BT04220531B GMS 2022-05-31 1005 25.05.–30.05. AfD 7
5 BT05220528Y INSA 2022-05-28 1337 23.05.–27.05. AfD 9
6 BT01170615B Infratest dimap 2017-06-15 1018 12.06.–13.06. AfD 8
7 BT03170701L Verian 2017-07-01 936 22.06.–24.06. AfD 7
8 BT13170623H YouGov 2017-06-23 1523 19.06.–22.06. AfD 7
```

Try it: how much did the polls agree right before the 2025 election?

Your task: the 2025 federal election was held 2025-02-23. Calculate the mean, variance, and standard deviation of the Green party's polled share in the four weeks leading up to it, then plot the distribution of those final polls against the Greens' actual result (11.6%).


```
gruene_pre_election <- polls_long |>
  filter(party == "Grüne", between(Date, as.Date("2025-01-26"), as.Date("2025-02-23")))
gruene_pre_election
```

>_ OUTPUT

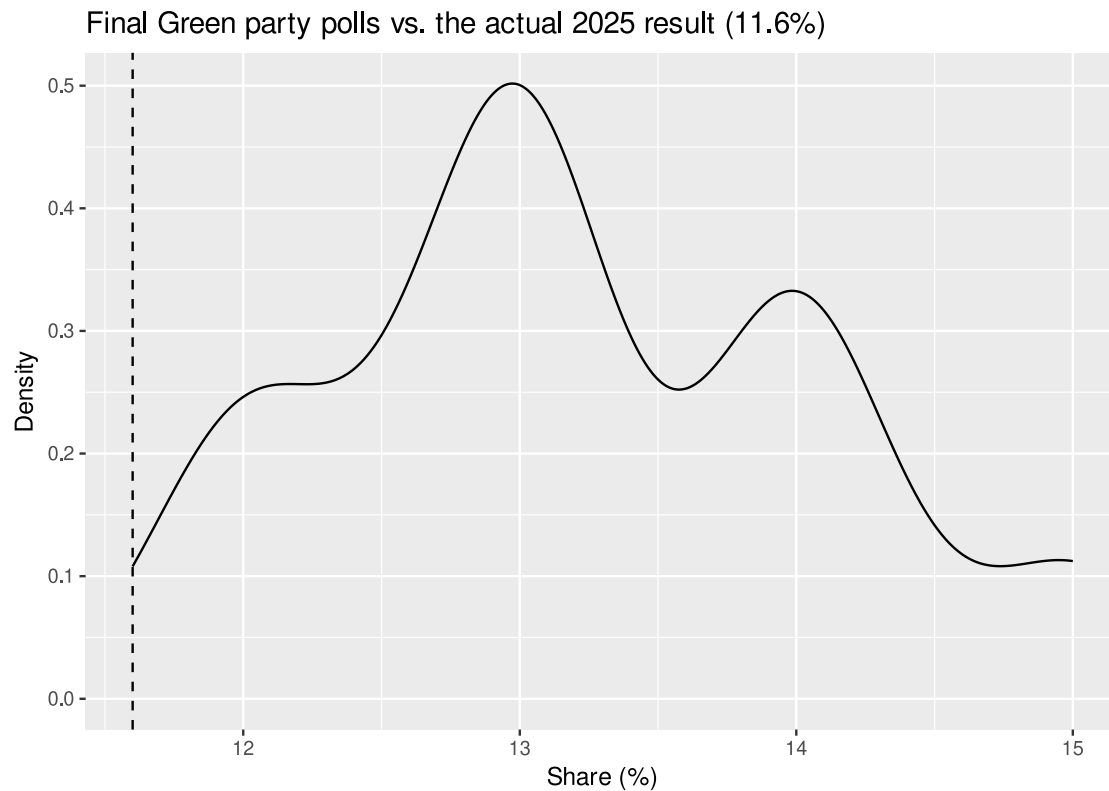
```
# A tibble: 33 × 7
  Poll_ID Pollster Date n Period party share
  <chr> <chr> <date> <dbl> <chr> <chr> <dbl>
1 BT05250127D INSA 2025-01-27 2006 24.01.-27.0... Grue... 12.5
2 BT02250128N Forsa 2025-01-28 2504 21.01.-27.0... Grue... 14
3 BT13250129C YouGov 2025-01-29 1781 24.01.-27.0... Grue... 13
4 BT06250130F Forschungsgruppe Wahlen 2025-01-30 1428 27.01.-29.0... Grue... 14
5 BT01250130Q Infratest dimap 2025-01-30 1336 27.01.-29.0... Grue... 15
6 BT05250130E INSA 2025-01-30 1001 30.01.-30.0... Grue... 13
7 BT03250131E Verian 2025-01-31 1461 22.01.-28.0... Grue... 14
8 BT05250201F INSA 2025-02-01 1203 30.01.-31.0... Grue... 12
9 BT05250203G INSA 2025-02-03 2004 31.01.-03.0... Grue... 13
10 BT02250204O Forsa 2025-02-04 2503 28.01.-03.0... Grue... 15
# i 23 more rows
```

```
gruene_pre_election |>
  summarize(
    mean_share = mean(share, na.rm = TRUE),
    var_share = var(share, na.rm = TRUE),
    sd_share = sd(share, na.rm = TRUE)
  )
```

>_ OUTPUT

```
# A tibble: 1 × 3
  mean_share var_share sd_share
  <dbl> <dbl> <dbl>
1 13.2 0.799 0.894
```

```
gruene_pre_election |>
  ggplot(aes(x = share)) +
  geom_density() +
  geom_vline(xintercept = 11.6, linetype = "dashed") +
  labs(title = "Final Green party polls vs. the actual 2025 result (11.6%)", x = "Share (%)", y = "Density")
```



Try it: BSW's rise above — and fall back below — the 5% threshold

Your task: BSW is a newly founded party — founded [2024-01-08](#) — that had to clear the 5% electoral threshold to enter parliament at all. Look at the remainder of that year (2024-01-08 to 2024-12-31): `mutate()` a `cleared_threshold` column. Compute the average share per pollster for each above and below the threshold. Plot the distribution of BSW's share by pollster.

```
bsw_2024 <- polls_long |>
  filter(party == "BSW", between(Date, as.Date("2024-01-08"), as.Date("2024-12-31")), !
is.na(share)) |>
  mutate(cleared_threshold = if_else(share >= 5, "above 5%", "below 5%"))
bsw_2024
```

> OUTPUT

```
# A tibble: 224 × 8
  Poll_ID      Pollster   Date      n Period party share cleared_threshold
  <chr>        <chr>    <date>    <dbl> <chr>  <chr>  <dbl> <chr>
1 BT06240112G Forschungs... 2024-01-12 1337 09.01... BSW      4 below 5%
2 BT05240120V INSA          2024-01-20 1203 15.01... BSW      7 above 5%
3 BT02240123K Forsa          2024-01-23 2503 16.01... BSW      3 below 5%
4 BT03240127P Verian          2024-01-27 1922 16.01... BSW      3 below 5%
5 BT05240127X INSA          2024-01-27 1201 22.01... BSW      7 above 5%
6 BT05240129Y INSA          2024-01-29 2002 26.01... BSW      7 above 5%
7 BT02240130L Forsa          2024-01-30 2506 23.01... BSW      3 below 5%
8 BT01240201B Infratest ... 2024-02-01 1303 29.01... BSW      5 above 5%
9 BT06240202H Forschungs... 2024-02-02 1217 30.01... BSW      6 above 5%
10 BT05240203A INSA          2024-02-03 1202 29.01... BSW      7 above 5%
# i 214 more rows
```

```
bsw_2024 |>
  group_by(Pollster, cleared_threshold) |>
  summarize(n = n(), avg_share = mean(share)) |>
  ungroup()
```

MESSAGE

`summarise()` has regrouped the output.
i Summaries were computed grouped by Pollster and cleared_threshold.
i Output is grouped by Pollster.
i Use `summarise(.groups = "drop\_last")` to silence this message.
i Use `summarise(.by = c(Pollster, cleared\_threshold))` for per-operation grouping (`?dplyr::dplyr\_by`) instead.

> OUTPUT

```
# A tibble: 11 × 4
  Pollster      cleared_threshold    n avg_share
  <chr>        <chr>          <int>    <dbl>
1 Allensbach    above 5%         11     7.5
2 Forsa         above 5%         37     6.16
3 Forsa         below 5%         13     3.85
4 Forschungsgruppe Wahlen above 5%         18     6.17
5 Forschungsgruppe Wahlen below 5%          1     4
6 GMS           above 5%          6     6.33
7 INSA          above 5%         99     7.93
8 Infratest dimap above 5%         14     6.21
9 Verian        above 5%         12     6.17
10 Verian        below 5%          2     3.5
11 YouGov        above 5%         11     7.64
```

```
bsw_2024 |>
  ggplot(aes(x = Pollster, y = share)) +
  geom_boxplot() +
  geom_hline(yintercept = 5, linetype = "dashed") +
  scale_x_discrete(guide = guide_axis(n.dodge = 2)) +
  labs(title = "BSW: polled share by pollster, 2024, vs. the 5% threshold", x = NULL,
y = "Share (%)")
```

